ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading Concepts in C++

Assignment Report– Bloom's Level K5

Name	S.THANISHKA
Register Number	192521457
Course / Section	DSA0119-OBJECT ORIENTED PROGRAMMING IN C++
Submission date	11/11/2026
Total marks	50

Q1 – Constructors and Constructor Overloading

1. Aim and requirements

Create a Book class for library records. It must support an empty record, a donated book with only a title and a fully specified book. Every constructor must use a member initializer list. Constructor delegation should avoid repeated initialization logic, and a negative number of copies must not be accepted.

2. C++ program

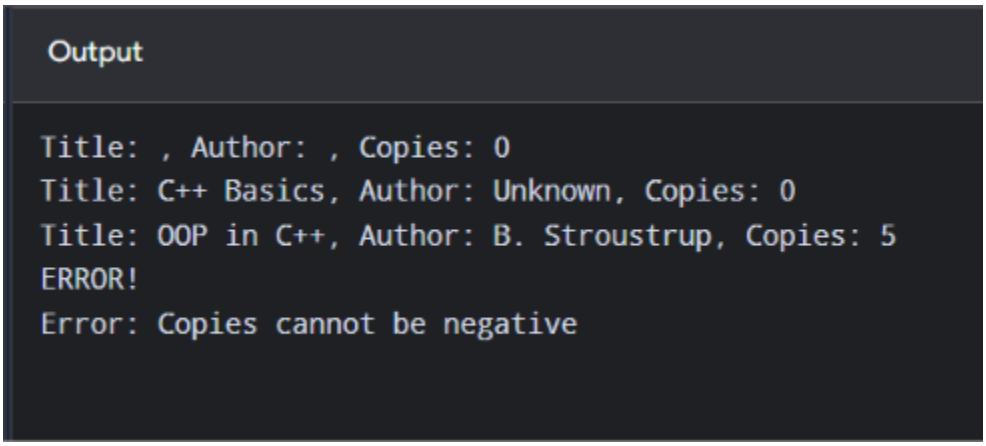
```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Book {
    string title;
    string author;
    int copies;
    static int validate(int n) {
        if (n < 0) throw invalid_argument("Copies cannot be negative");
        return n;
    }
public:
    Book() : Book("", "", 0) {}
    explicit Book(const string& t) : Book(t, "Unknown", 0) {}
    Book(const string& t, const string& a, int n)
        : title(t), author(a), copies(validate(n)) {}

    void display() const {
        cout << "Title: " << title << ", Author: " << author
             << ", Copies: " << copies << '\n';
    }
};
```

```
int Book::copies; // not required; no static data definition is needed
int main() {
    try {
        Book empty;
        Book donated("C++ Basics");
        Book full("OOP in C++", "B. Stroustrup", 5);
        empty.display();
        donated.display();
        full.display();
        Book invalid("Invalid", "Author", -2);
    } catch (const exception& e) {
        cout << "Error: " << e.what() << '\n';
    }
}
```

Expected output:



Title: , Author: , Copies: 0

Title: C++ Basics, Author: Unknown, Copies: 0

Title: OOP in C++, Author: B. Stroustrup, Copies: 5

Error: Copies cannot be negative

3. Design and analysis

The compiler supplies a default constructor only when the class has no user-declared constructor. If any constructor is declared by the programmer, the compiler does not automatically provide a

no-argument default constructor. A default constructor can be written explicitly or generated with = default.

A member initializer list initializes data members directly before the constructor body runs. Assignment inside the body first default-constructs the members and then assigns new values. Direct initialization is generally more efficient and is required for const members, reference members and members without a default constructor.

Members are initialized in the order in which they are declared in the class, not the order written in the initializer list. The initializer list should follow declaration order to avoid confusion and -Wreorder warnings.

Object states after construction: empty = (title="", author="", copies=0); donated = (title="C++ Basics", author="Unknown", copies=0); full = (title="OOP in C++", author="B. Stroustrup", copies=5).

4. Experimental investigation and design question

Removing the default constructor and then declaring Book books[3]; causes a compilation error if no accessible default constructor exists, because each array element must be default-constructed. A -Wreorder warning can be demonstrated by deliberately listing members in an order different from their declaration order; correct the list afterward.

The negative-copy test above throws invalid_argument and prevents an invalid object from being created. The explicit keyword prevents unintended implicit conversion from string to Book. For example, with a non-explicit one-argument constructor, Book b = string("Novel"); may compile; with explicit, copy-initialization is rejected, while Book b("Novel"); remains valid.

5. Alternative concept and complexity

A single constructor such as Book(string t="", string a="Unknown", int n=0) can replace the overload set. It still validates n once, but the call sites are less descriptive and default arguments can make the interface less clear. Constructor delegation centralizes validation and initialization. Creating one Book takes O(1) time and O(1) extra space in either design. Both designs should validate the copy count in one common implementation path.

Q2 – Copy Constructors: Shallow and Deep Copies

1. Aim and requirements

A Prescription owns a dynamically allocated array of medicine names. Implement a deep-copying copy constructor, copy assignment operator and destructor so that each object owns an independent array. Test copying, assignment, self-assignment and sanitizer behaviour.

2. Complete-the-code solution

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
   int n;
   string* medicines;
public:
   explicit Prescription(int count)
      : n(count), medicines(count > 0 ? new string[count] : nullptr) {}

   Prescription(const Prescription& other)
      : n(other.n),
        medicines(other.n > 0 ? new string[other.n] : nullptr) {
      for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
   }

   Prescription& operator=(const Prescription& other) {
      if (this != &other) {
         string* temp = other.n > 0 ? new string[other.n] : nullptr;
         for (int i = 0; i < other.n; ++i) temp[i] = other.medicines[i];
         delete[] medicines;
         medicines = temp;
         n = other.n;
      }
```

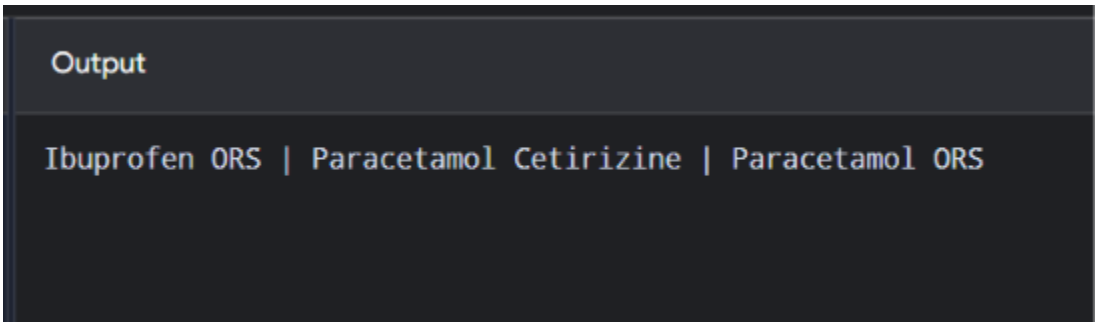
```
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const string& m) { medicines[i] = m; }
    string get(int i) const { return medicines[i]; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output:



Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

3. Shallow copy versus deep copy

A shallow copy copies the pointer value. The original and copied objects then point to the same array. Changing one object changes what the other observes. When both destructors execute, both attempt to delete the same allocation, causing a double-free error or undefined behaviour.

A deep copy allocates a separate array and copies each medicine name. The two objects have independent storage, so modifying one does not change the other. Each destructor releases only its own array.

Memory diagram: Shallow: p.medicines ─┐ and q.medicines ──┴→ one shared array. Deep: p.medicines → Array A; q.medicines → Array B, with equal initial contents but different addresses.

4. Rule of Three and copy constructor calls

The Rule of Three states that if a class needs a user-defined destructor, copy constructor or copy assignment operator because it manages a resource, it usually needs all three. They work together to define ownership, copying and cleanup safely.

A copy constructor is called when an object is initialized from another object of the same class, when an object is passed by value, and when an object is returned by value in situations where copy elision does not remove the copy. Passing by const reference does not create a copy.

Self-assignment p = p is protected by checking this != &other. The temporary array is allocated and copied before the old array is released, providing safer assignment if allocation fails.

5. Experimental investigation

Add a print statement inside the copy constructor to count copies. Pass Prescription by value and then by const reference; by-value normally makes a copy, while const-reference avoids that parameter copy. Measure 10, 1,000 and 100,000 medicines with a steady clock and record your own elapsed times. Run both shallow and deep versions with AddressSanitizer. The shallow version may report double-free/use-after-free; the deep version should complete without memory errors.

6. Alternative concept and complexity

Using vector<string> removes manual new/delete and allows the compiler-generated copy operations to perform an independent element-wise copy. This is the Rule of Zero: let standard library types manage resources. Copying n medicine names takes O(n) time and O(n) additional space for both vector and a correctly implemented dynamic array. A shallow pointer copy is O(1), but it is unsafe because ownership is shared incorrectly.

Q3 – Dynamic Constructors and Destructors

1. Aim and requirements

Create an auditorium seat map whose row and seat counts are known at runtime. Allocate the map dynamically, support booking and cancellation, release every allocation in the destructor and print a closing summary.

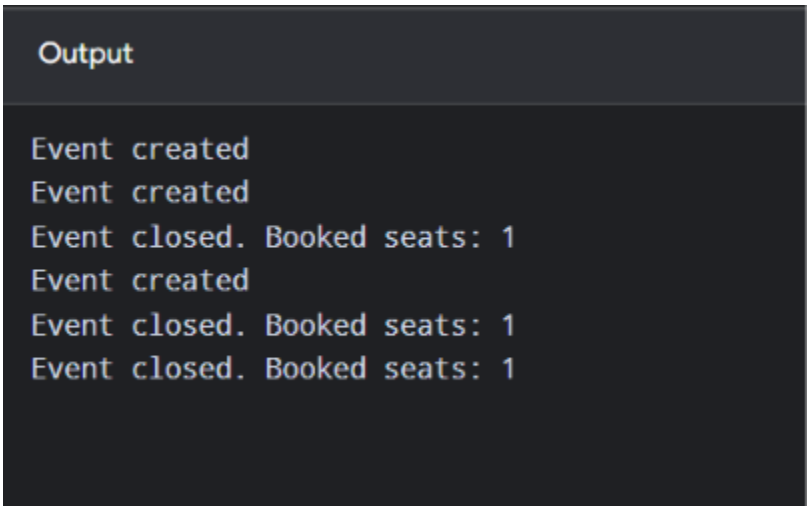
2. C++ program

```
#include <iostream>
using namespace std;

class Event {
   int rows, seats, booked;
   char** map;
public:
   Event(int r, int s) : rows(r), seats(s), booked(0), map(nullptr) {
      if (rows > 0 && seats > 0) {
         map = new char*[rows];
         for (int i = 0; i < rows; ++i) {
            map[i] = new char[seats];
            for (int j = 0; j < seats; ++j) map[i][j] = 'O';
         }
      }
      cout << "Event created\n";
   }
   bool book(int r, int s) {
      if (!map || r < 0 || r >= rows || s < 0 || s >= seats ||
         map[r][s] == 'X') return false;
      map[r][s] = 'X'; ++booked; return true;
   }
   bool cancel(int r, int s) {
      if (!map || r < 0 || r >= rows || s < 0 || s >= seats ||
         map[r][s] == 'O') return false;
```

```
        map[r][s] = 'O'; --booked; return true;
    }
    ~Event() {
        if (map) {
            for (int i = 0; i < rows; ++i) delete[] map[i];
            delete[] map;
        }
        cout << "Event closed. Booked seats: " << booked << '\n';
    }
};
int main() {
    Event local(2, 3);
    local.book(0, 1);
    {
        Event blockEvent(1, 2);
        blockEvent.book(0, 0);
    }
    Event* heapEvent = new Event(2, 2);
    heapEvent->book(1, 1);
    delete heapEvent;
}
```

Illustrative output (destructor order follows object lifetime):



Event created

Event created

Event closed. Booked seats: 1

Event created

Event closed. Booked seats: 1

Event closed. Booked seats: 1

3. Design and analysis

A local object is destroyed automatically when its scope ends. A block-scope object is destroyed at the closing brace of its block. A dynamically allocated object remains alive until delete is called. Objects in the same scope are normally destroyed in reverse order of construction.

For char** allocation, first allocate an array of row pointers, then allocate one character array per row. Correct cleanup deletes each row with delete[] and then deletes the pointer array with delete[]. Forgetting delete causes a memory leak. Using delete instead of delete[] for an array is undefined behaviour.

A destructor has no return type and takes no arguments. It cannot be overloaded because destruction has one fixed role per object: release its resources when its lifetime ends.

Memory layout: map points to a row-pointer array of length rows; each pointer points to a separate character array of length seats. Total seat storage is rows × seats characters, plus rows pointers.

4. Experimental investigation and RAII

Print messages in constructor and destructor to record lifetime order. Run once with a deliberately omitted delete and once with correct cleanup under -fsanitize=address. Record the actual report. Seat storage for 10×20 is 200 bytes, 100×200 is 20,000 bytes, and 1,000×2,000 is 2,000,000 bytes, excluding row pointers and allocator overhead. A zero-row event should allocate no row map and should safely reject booking.

RAII means resource acquisition is tied to object construction and resource release to destruction. This reduces forgotten cleanup and helps ensure cleanup happens on normal scope exit.

5. Predict-the-output answer

+A +B -B copyAc use -Ac +P -P end -A

A is constructed first. B is constructed inside the nested block and destroyed at the closing brace. Passing a to use(Logger) passes by value, so a copy named Ac is created; that parameter is destroyed when use returns. P is created dynamically and destroyed by delete p. The text end is printed, and finally a is destroyed as main exits.

6. Alternative concept and complexity

A single contiguous allocation stores rows × seats characters and accesses an element using index r × seats + s. A vector<vector<char>> stores a vector of row vectors. Access is O(1) in all three layouts. The row-pointer design uses rows + 1 allocations; a contiguous block uses one allocation and avoids row pointers. A vector of vectors generally uses one allocation for the outer vector plus one allocation per row, subject to implementation details. Contiguous storage improves locality and reduces allocation overhead.

Q4 – Overloading Arithmetic and Stream Operators

1. Aim and requirements

Implement Money using an integer number of paise. Overload addition, subtraction, compound addition, equality and less-than comparison, plus stream insertion and extraction. Integer storage avoids common floating-point rounding errors in currency arithmetic.

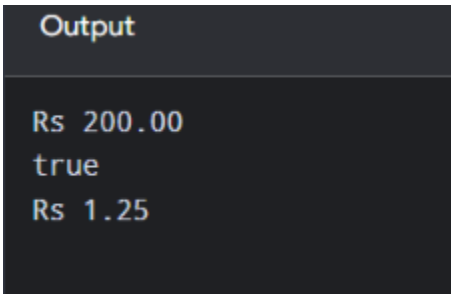
2. C++ program

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

class Money {
   long long paise;
public:
   Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
   Money operator+(const Money& other) const {
      Money result; result.paise = paise + other.paise; return result;
   }
   Money operator-(const Money& other) const {
      Money result; result.paise = paise - other.paise; return result;
   }
   Money& operator+=(const Money& other) {
      paise += other.paise; return *this;
   }
   bool operator==(const Money& other) const { return paise == other.paise; }
   bool operator<(const Money& other) const { return paise < other.paise; }
   friend ostream& operator<<(ostream& out, const Money& m) {
      long long a = llabs(m.paise);
      if (m.paise < 0) out << "-";
      return out << "Rs " << a / 100 << '.' << setw(2)
                 << setfill('0') << a % 100 << setfill(' ');
   }
   friend istream& operator>>(istream& in, Money& m) {
      long long r; int p;
      if (in >> r >> p) m.paise = r * 100 + p;
      return in;
   }
```

```
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';
    cout << (Money(0,75) + Money(0,50)) << '\n';
}
```

Expected output:



Rs 200.00
true
Rs 1.25

3. Design and analysis

The + operator creates and returns a new Money value, leaving its operands unchanged. The += operator modifies the left operand and returns Money& to *this, allowing chained operations without creating an unnecessary result object.

Arithmetic and comparison operators can be members because they operate naturally on the left-hand Money object. The stream insertion operator is usually a non-member friend because the left operand is ostream, not Money. A friend function can access the private paise value while allowing cout << money.

The expression a + b is interpreted as a.operator+(b) when + is a member operator. A non-member form is operator+(a, b).

4. Experimental investigation and engineering decision

Rs 0.75 + Rs 0.50 produces Rs 1.25 because 75 + 50 = 125 paise. Subtraction may produce a negative balance; the class should define and document whether negative values are allowed. The

example prints a leading minus sign. Compare readability by writing the same bill calculation using add(a,b) and using a+b. For a million additions, benchmark integer-paise and double-based versions on the same compiler and optimization level; report measured values rather than assumed values.

Multiplying one Money amount by another Money amount is generally not meaningful because the result has squared currency units. Multiplying Money by a scalar quantity (for example, number of items) can be meaningful if rounding rules are defined.

5. Alternative concept and complexity

Operators can be implemented as non-member friend functions or replaced by named functions such as add() and equals(). All basic arithmetic and comparison operations take O(1) time and O(1) extra space. A non-member operator can support scalar-on-left expressions such as 100 + m if a suitable conversion or overload exists; a member operator on Money cannot be called when the left operand is an int.

Q5 – Increment, Comparison and Subscript Operators

1. Aim and requirements

Implement a Time class with prefix and postfix increment, equality and less-than comparison. Implement a Playlist subscript operator in const and non-const forms with range checking. Test negative and too-large indices, increment values and time rollover.

2. C++ program

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

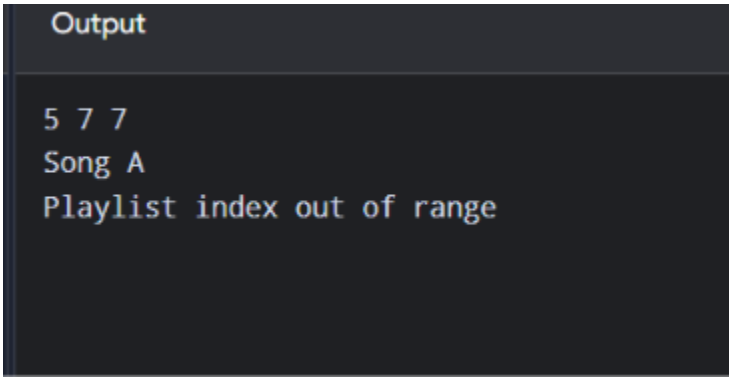
class Time {
   int seconds;
public:
   explicit Time(int s = 0) : seconds(s) {}
   Time& operator++() { ++seconds; return *this; }
   Time operator++(int) { Time old(*this); ++seconds; return old; }
   bool operator==(const Time& t) const { return seconds == t.seconds; }
   bool operator<(const Time& t) const { return seconds < t.seconds; }
   int value() const { return seconds; }
};

class Playlist {
   vector<string> tracks;
public:
   Playlist(initializer_list<string> x) : tracks(x) {}
   string& operator[](int i) {
      if (i < 0 || i >= (int)tracks.size())
         throw out_of_range("Playlist index out of range");
      return tracks[i];
   }
   const string& operator[](int i) const {
      if (i < 0 || i >= (int)tracks.size())
         throw out_of_range("Playlist index out of range");
      return tracks[i];
   }
```

```
    int size() const { return (int)tracks.size(); }
};

int main() {
    Time t(5);
    Time a = t++;
    Time b = ++t;
    cout << a.value() << ' ' << b.value() << ' ' << t.value() << '\n';
    Playlist p{"Song A", "Song B"};
    cout << p[0] << '\n';
    try { cout << p[p.size()] << '\n'; }
    catch (const out_of_range& e) { cout << e.what() << '\n'; }
}
```

Expected output:



5 7 7

Song A

Playlist index out of range

3. Design and analysis

The compiler distinguishes prefix ++i from postfix i++ by the dummy int parameter in operator++(int). Prefix increments first and returns a reference to the updated object. Postfix saves the old value, increments the object and returns the saved copy. Therefore postfix usually performs an extra copy for class types.

A non-const operator[] returns a modifiable reference so a track can be changed. A const operator[] returns a const reference so read-only access works on const Playlist objects without allowing

modification. Out-of-range access should be handled consistently; this solution throws std::out_of_range.

For a = t++, a receives the old value 5 and t becomes 6. Then b = ++t increments t to 7 and b receives 7. Thus the output is 5 7 7.

4. Experimental investigation and conclusion

Test playlist[-1] and playlist[size]; both should throw out_of_range. For a full clock representation, implement carry logic so 59:59 advances to 1:00:00. Benchmark 10,000,000 prefix and postfix increments at -O0 and -O2 using the same test and compiler; optimization may remove much of the apparent difference, so use observable results and repeat measurements.

Programmers often prefer ++i for class types because it avoids constructing and returning the old-value copy required by postfix increment when that old value is not needed.

5. Alternative concept and complexity

An at(i) member function can throw std::out_of_range, similar to this checked operator[]. An unchecked operator[] is faster in principle but invalid indexing causes undefined behaviour. Access is O(1) in all cases; checked access adds a bounds comparison. Benchmark ten million accesses to estimate the real cost on the target machine.

Q6 – The Rules of Operator Overloading

1. Aim and requirements

Create a Text class that owns a dynamically allocated null-terminated character array. Implement constructor, copy constructor, copy assignment, destructor, concatenation, equality, inequality and stream output. Ensure deep copying and safe self-assignment.

2. C++ program

```
#include <iostream>
#include <cstring>
```

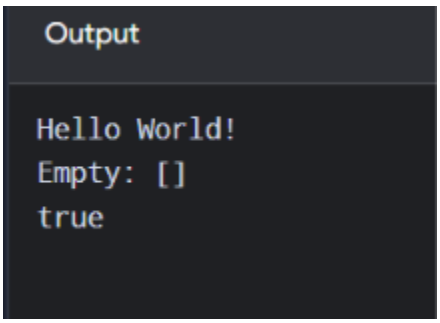
```
using namespace std;

class Text {
   char* data;
public:
   Text(const char* s = "") {
      data = new char[strlen(s) + 1];
      strcpy(data, s);
   }
   Text(const Text& other) {
      data = new char[strlen(other.data) + 1];
      strcpy(data, other.data);
   }
   Text& operator=(const Text& other) {
      if (this != &other) {
         char* temp = new char[strlen(other.data) + 1];
         strcpy(temp, other.data);
         delete[] data;
         data = temp;
      }
      return *this;
   }
   ~Text() { delete[] data; }

   Text operator+(const Text& other) const {
      size_t n = strlen(data) + strlen(other.data);
      char* temp = new char[n + 1];
      strcpy(temp, data);
      strcat(temp, other.data);
      Text result(temp);
      delete[] temp;
      return result;
   }
```

```
    bool operator==(const Text& other) const {
        return strcmp(data, other.data) == 0;
    }
    bool operator!=(const Text& other) const { return !(*this == other); }
    friend ostream& operator<<(ostream& out, const Text& t) {
        return out << t.data;
    }
};
int main() {
    Text a("Hello "), b("World"), c("!");
    Text message = a + b + c;
    cout << message << '\n';
    Text empty;
    cout << "Empty: [" << empty << "]\n";
    message = message;
    cout << boolalpha << (a != b) << '\n';
}
```

Expected output:



Hello World!

Empty: []

true

3. Rules of operator overloading

The operators that cannot be overloaded are ::, ., .*, ?: and sizeof. Operator overloading cannot change an operator's precedence, associativity or number of operands (arity). At least one operand must be a user-defined type; operators cannot be redefined for two built-in types alone.

operator= must be a non-static member function. A friend is useful when an operator needs access to private data but the left operand is not the class type, as with stream insertion. Friendship should be limited to cases where it is genuinely needed.

The Rule of Three applies because Text manages a raw dynamic allocation. The copy constructor and assignment operator must allocate independent character arrays, and the destructor must release the array. The assignment operator checks self-assignment.

4. Experimental investigation and conclusion

Count copies by adding diagnostic output to the copy constructor. The expression a + b + c is left associative and is evaluated as (a + b) + c. Each concatenation creates a combined result and may create temporary objects; copy elision can affect observed copy counts. Run with AddressSanitizer and verify that no leaks, double frees or invalid accesses are reported. Compare 100,000 concatenations with std::string using identical compiler settings and input sizes.

Writing Text demonstrates ownership, deep copying, exception safety and operator design. In normal applications, std::string is preferable because it is tested, optimized and handles memory management. A custom class is justified for specialized storage, embedded systems, educational purposes or unusual constraints.

5. Alternative concept and complexity

Replacing Text with std::string applies the Rule of Zero and removes manual memory management. Another implementation can define operator+ by taking the left operand by value, appending the right operand with += and returning the result. For lengths n1, n2 and n3, evaluating a+b+c copies characters proportional to the intermediate and final string lengths; a straightforward two-stage concatenation is O(n1+n2+n3) overall in character work, though allocation and temporary counts depend on implementation. Temporary storage is proportional to the intermediate result and final result. Benchmark the actual implementation rather than assuming a fixed number of copies.

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Interpret the problem

The data are the real and imaginary components of impedance, represented as a + bj. Required operations are addition, subtraction, multiplication, division, equality, printing in rectangular form and magnitude calculation. Constructors required are a default constructor, a constructor from a real component and a constructor from real and imaginary components.

Measurable requirements: each arithmetic operation returns the mathematically correct result; magnitude equals sqrt(real² + imaginary²); division by zero is detected; output displays both components; series impedance is the sum of impedances; parallel impedance is (Z1 × Z2) / (Z1 + Z2).

Stage 2 – Design and formulas

Represent a complex value as (a,b), meaning a + bj. Addition: (a,b)+(c,d)=(a+c,b+d). Subtraction: (a,b)-(c,d)=(a-c,b-d). Multiplication: (a,b)(c,d)=(ac-bd, ad+bc). Division: (a,b)/(c,d)=((ac+bd)/(c^2+d^2), (bc-ad)/(c^2+d^2)); reject c^2+d^2=0. Magnitude: sqrt(a^2+b^2).

Arithmetic operators can be member functions. Stream insertion is a friend non-member because ostream is the left operand. Equality can be a member or non-member; exact floating-point equality is not always suitable for measured values, so a tolerance-based comparison may be preferable.

Stage 3 – Implementation and verification

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

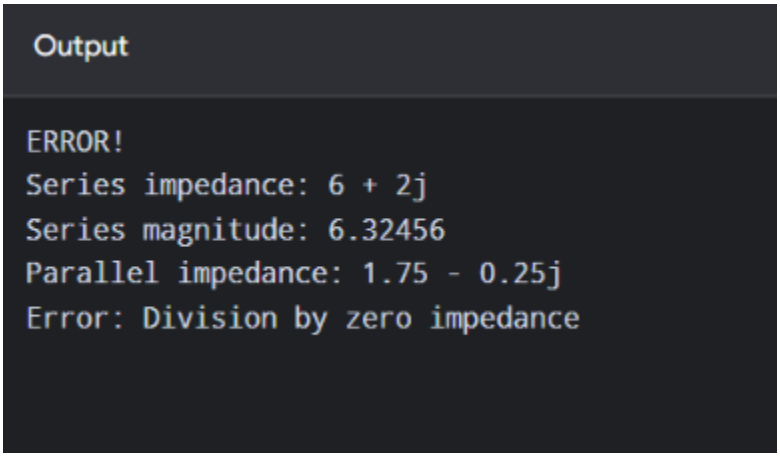
class Complex {
   double re, im;
public:
   Complex() : re(0), im(0) {}
   Complex(double real) : re(real), im(0) {}
   Complex(double real, double imag) : re(real), im(imag) {}

   Complex operator+(const Complex& z) const {
      return Complex(re + z.re, im + z.im);
   }
   Complex operator-(const Complex& z) const {
      return Complex(re - z.re, im - z.im);
   }
   Complex operator*(const Complex& z) const {
      return Complex(re*z.re - im*z.im, re*z.im + im*z.re);
   }
   Complex operator/(const Complex& z) const {
      double d = z.re*z.re + z.im*z.im;
```

```
        if (d == 0.0) throw runtime_error("Division by zero impedance");
        return Complex((re*z.re + im*z.im)/d,
                       (im*z.re - re*z.im)/d);
    }
    bool operator==(const Complex& z) const {
        const double eps = 1e-9;
        return fabs(re-z.re) < eps && fabs(im-z.im) < eps;
    }
    double magnitude() const { return sqrt(re*re + im*im); }
    friend ostream& operator<<(ostream& out, const Complex& z) {
        return out << z.re << (z.im < 0 ? " - " : " + ")
                   << fabs(z.im) << "j";
    }
};

int main() {
    try {
        Complex z1(4, 3), z2(2, -1);
        Complex series = z1 + z2;
        Complex parallel = (z1 * z2) / (z1 + z2);
        cout << "Series impedance: " << series << '\n';
        cout << "Series magnitude: " << series.magnitude() << '\n';
        cout << "Parallel impedance: " << parallel << '\n';
        Complex zero;
        Complex invalid = z1 / zero;
        cout << invalid << '\n';
    } catch (const exception& e) {
        cout << "Error: " << e.what() << '\n';
    }
}
```

Expected output:



Series impedance: 6 + 2j

Series magnitude: 6.32456

Parallel impedance: 1.75 - 0.25j

Error: Division by zero impedance

Hand verification: z1=4+3j and z2=2−j. Series = 6+2j; magnitude = sqrt(6^2+2^2)=sqrt(40)≈6.32456. Product z1z2=(4+3j)(2−j)=11+2j. Sum is 6+2j. Parallel = (11+2j)/(6+2j) = (70−10j)/40 = 1.75−0.25j.

Stage 4 – Defend the design

std::complex<double> already provides standard complex arithmetic and is preferable in production code because it is tested and maintained. The custom class demonstrates the formulas and allows custom output and explicit division-by-zero handling. Both designs store two double values and perform basic operations in O(1) time and O(1) space. Floating-point rounding remains a limitation, and exact equality is not generally reliable for computed values.

Alternative concept and complexity

Using std::complex<double> replaces the custom arithmetic implementation with the standard library type. Each basic operation remains O(1) time and O(1) space, storing two double components. The standard type reduces implementation work, but the application must still decide how to detect or handle a zero denominator and how to format output. A custom class gives direct control over those policies.